

IMPLEMENTACIÓN Y ANÁLISIS DE ALGORITMOS DE BÚSQUEDA Y ORDENAMIENTO EN JAVA

Introducción

Organizar y buscar datos de manera rápida y precisa es un trabajo fundamental en el ámbito de la programación. Los algoritmos de búsqueda y ordenamiento se implementan en muchos procesos informáticos desde motores de búsqueda hasta sistemas de inteligencia artificial. Por lo tanto, conocer y dominar algoritmos que permitan organizar y buscar datos de manera rápida y precisa es una habilidad indispensable para cualquier programador.

En el presente proyecto se implementará de forma práctica algoritmos de búsqueda secuencial y binaria, junto con los métodos de ordenamiento burbuja, inserción y selección; con el fin de comprender su funcionamiento, analizar su rendimiento y aplicar principios de diseño en su implementación.

Objetivos del proyecto

- Implementar y analizar cinco algoritmos de búsqueda y ordenamiento en Java con el fin de comprender su funcionamiento
- Realizar pruebas cronometradas para probar la eficiencia de cada algoritmo con un conjunto de datos pequeños, mediano y grande.
- Presentar los resultados en gráficas ilustrativas y tablas resúmenes para facilitar su análisis y evidenciar sus diferencias de rendimiento.

Algoritmos implementados

- Algoritmos de búsqueda:

- Búsqueda binaria

Este algoritmo requiere que la lista o el arreglo de elementos esté ordenado. Su funcionamiento compara el valor buscado con el elemento del medio, si este es igual termina el proceso. Si el valor es menor, busca en la mitad izquierda y si es mayor en la mitad derecha. Este proceso se repite dividiendo el espacio de búsqueda a la mitad cada vez.

Ejemplo de código

```
//recibe un arreglo ordenado, el índice inicial (x), el índice final (sizeArreglo)
y el número a buscar
public int busquedaBinaria(int arreglo[], int x, int sizeArreglo, int numero) {

    while (x <= sizeArreglo) {
        // Calcula el índice medio
        int indice = (x + sizeArreglo) / 2;
        // Verifica si el número está en el índice medio
        if (arreglo[indice] == numero) {
            return indice; // Retorna el índice del número encontrado
        } else if (arreglo[indice] > numero) {
            sizeArreglo = indice - 1; // Busca en la mitad izquierda
        } else {
            x = indice + 1; // Busca en la mitad derecha
        }
    }
    return -1; // Retorna -1 si el número no se encuentra en el arreglo
}
```

- Búsqueda secuencial

Este algoritmo recorre una lista o arreglo de elementos y va comparando cada elemento con el valor buscado. Si lo encuentra, este puede devolver la posición o el dato según se programe. Además, si llega hasta el final sin encontrarlo, indicará que no se encuentra entre los elementos.

Ejemplo de código

```
public static int busquedaSecuencial(int arreglo[], int numero) {
    // Verifica si el arreglo es nulo o tiene longitud cero
    int n = arreglo.length;

    for (int i = 0; i < n; i++) {
        // Si el elemento en la posición i es igual al número buscado, retorna la posición
        if (arreglo[i] == numero)
            return i;
    }
    return -1; // Retorna -1 si el número no se encuentra en el arreglo
}
```

- Algoritmos de ordenamiento

- Ordenamiento burbuja

Este algoritmo compara dos elementos que estén a la par y los intercambia si están ubicados en el orden incorrecto. Al terminar cada ronda, el elemento más grande pasa al final del arreglo de elementos. Este proceso se repite hasta que ya no se puedan hacer intercambios, es decir que todos los elementos estén ordenados.

Ejemplo de código

```
public static void ordenarBurbuja(int arreglo[], int sizeArreglo) {
    boolean intercambio;
    int auxiliar;

    for (int i = 0; i < sizeArreglo - 1; i++) {
        intercambio = false; // Inicializa la variable de intercambio como falsa
        for (int j = 0; j < sizeArreglo - i - 1; j++) {
            // Compara los elementos del arreglo
            if (arreglo[j] > arreglo[j + 1]) {
                // Intercambia los elementos si están en el orden incorrecto
                auxiliar = arreglo[j]; // Guarda el valor del elemento actual
                arreglo[j] = arreglo[j + 1]; // Asigna el valor del siguiente elemento al actual
                arreglo[j + 1] = auxiliar; // Asigna el valor guardado al siguiente elemento
                intercambio = true; // Marca que se ha realizado un intercambio
            }
        }
        if (intercambio == false) { // Si no se realizaron intercambios, el arreglo ya está
            // ordenado por lo que se puede salir del bucle
            break;
        }
    }
}
```

- Ordenamiento por inserción

El funcionamiento de este algoritmo es más sencillo ya que toma cada elemento del conjunto y lo inserta en la posición correcta dentro de la parte ya ordenada. Es decir, si el primer elemento es 4, al tomar el segundo elemento 3 lo insertará antes del 4 y así sucesivamente con el conjunto de elementos.

Ejemplo de código

```
public static void ordenamientoInsercion(int arr[], int arregloSize) {  
    for (int i = 1; i < arregloSize; ++i) {  
        int indice = arr[i];  
        int posicion = i - 1;  
  
        // Mueve los elementos del arreglo que son mayores que el índice a una posición adelante  
        while (posicion >= 0 && arr[posicion] > indice) {  
            // Mueve el elemento a la derecha  
            arr[posicion + 1] = arr[posicion];  
            posicion = posicion - 1; // Disminuye la posición  
        }  
        arr[posicion + 1] = indice; // Inserta el índice en la posición correcta  
    }  
}
```

- Ordenamiento por selección

Este algoritmo busca el mínimo elemento y lo coloca en la primera posición, continua con siguientes mínimos y lo coloca en la segunda posición, así continúa este ciclo hasta ordenarlo todo.

Ejemplo de código

```
public static void OrdenamientoSeleccion(int[] arreglo, int sizeArreglo) {  
    for (int i = 0; i < sizeArreglo - 1; i++) {  
        int minimo_indice = i; //Inicializa el índice del elemento mínimo como el índice actual  
  
        // Encuentra el índice del elemento mínimo en el subarreglo no ordenado  
        for (int j = i + 1; j < sizeArreglo; j++) {  
            if (arreglo[j] < arreglo[minimo_indice]) { //Compara el elemento actual con el mínimo encontrado  
                minimo_indice = j; //Actualiza el índice del mínimo si se encuentra un elemento menor  
            }  
        }  
  
        int auxiliar = arreglo[i]; // Guarda el valor del elemento actual  
        arreglo[i] = arreglo[minimo_indice]; // Asigna el valor del elemento mínimo al elemento actual  
        arreglo[minimo_indice] = auxiliar; // Asigna el valor guardado al elemento mínimo encontrado  
    }  
}
```

Análisis de complejidad Big O

- Algoritmos de búsqueda:

- Búsqueda secuencial:

Debido a la lógica de este algoritmo, el mejor caso el valor buscado se encuentra en la mejor posición, es decir que solo se hará 1 comparación. Pero en el peor de los casos, el algoritmo tendrá que hacer n comparaciones para encontrarlo y si la lista crece, se harán más comparaciones y el tiempo de ejecución aumentará.

Tiempo	Mejor caso	Peor caso	Caso Promedio
	$O(1)$	$O(n)$	$O(n)$

- Búsqueda Binaria

La complejidad de este algoritmo es mayor ya que en cada ronda divide la lista a la mitad, por lo que, número de iteración estará dado por $\log_2(n)$. Por lo que su tiempo y complejidad aumentará dependiendo del tamaño de la lista, por ejemplo:

- En lista de 8, se harán 3 pasos como máximo:
 $\log_2 8 = 3 \rightarrow 2^3 = 8$.
- En una lista de 1000, se harán como máximo 10 pasos:
 $\log_2 1000 = 10 \rightarrow 2^{10} = 1000$.

Tiempo	Mejor caso	Peor caso	Caso Promedio
	$O(1)$	$O(\log n)$	$O(\log n)$

- Algoritmos de ordenamiento:

- Ordenamiento por burbuja:

Este es uno de los algoritmos poco eficientes debido a su mecánica de implementación y a su complejidad. Debido a que implementa dos bucles anidados, el tiempo crece a n iteraciones

al cuadrado y al hacer las comparaciones se da por la razón:
 $\frac{n*(n-1)}{2} \rightarrow O(n^2)$.

Tiempo	Mejor caso	Peor caso
	$O(n)$	$O(n^2)$

- Ordenamiento por inserción:

Debido a la mecánica de este algoritmo, se pueden dar dos casos para su implementación:

- En el peor caso la lista de elementos estará en orden inverso, por lo que mueve cada elemento $\rightarrow \frac{n^2}{2}$ movimientos.
- En el mejor caso la lista está casi ordenada, solo hace n comparaciones.

Tiempo	Mejor caso	Peor caso	Caso Promedio
	$O(n)$	$O(n^2)$	$O(n^2)$

- Ordenamiento por selección:

Este tipo de algoritmos tiene un tiempo fijo de implementación ya que siempre hace $\frac{n^2}{2}$ comparaciones, aunque la lista esté ordenada por lo que los intercambios tienen siempre un máximo n.

Tiempo	Siempre
	$O(n^2)$

Resultados de pruebas de rendimiento

Para realizar estas pruebas utilice el método `System.nanoTime()`, cree dos variables, inicio y final, para medir el tiempo de ejecución justo antes de llamar a la clase que contiene al algoritmo hasta el final de la ejecución, así como se muestra a continuación:

```
//Implementacion de algoritmo de ordenamiento por seleccion
OrdenamientoPorSeleccion ordenSeleccion = new OrdenamientoPorSeleccion();

long inicio = System.nanoTime();
ordenSeleccion.OrdenamientoSeleccion(arregloDesordenadoBig, sizeArreglo);

System.out.print("\nArreglo ordenado con el algoritmo de ordenamiento por seleccion: ");
for (int i = 0; i < sizeArreglo; i++) {
    System.out.print(arregloDesordenadoBig[i] + " ");
}

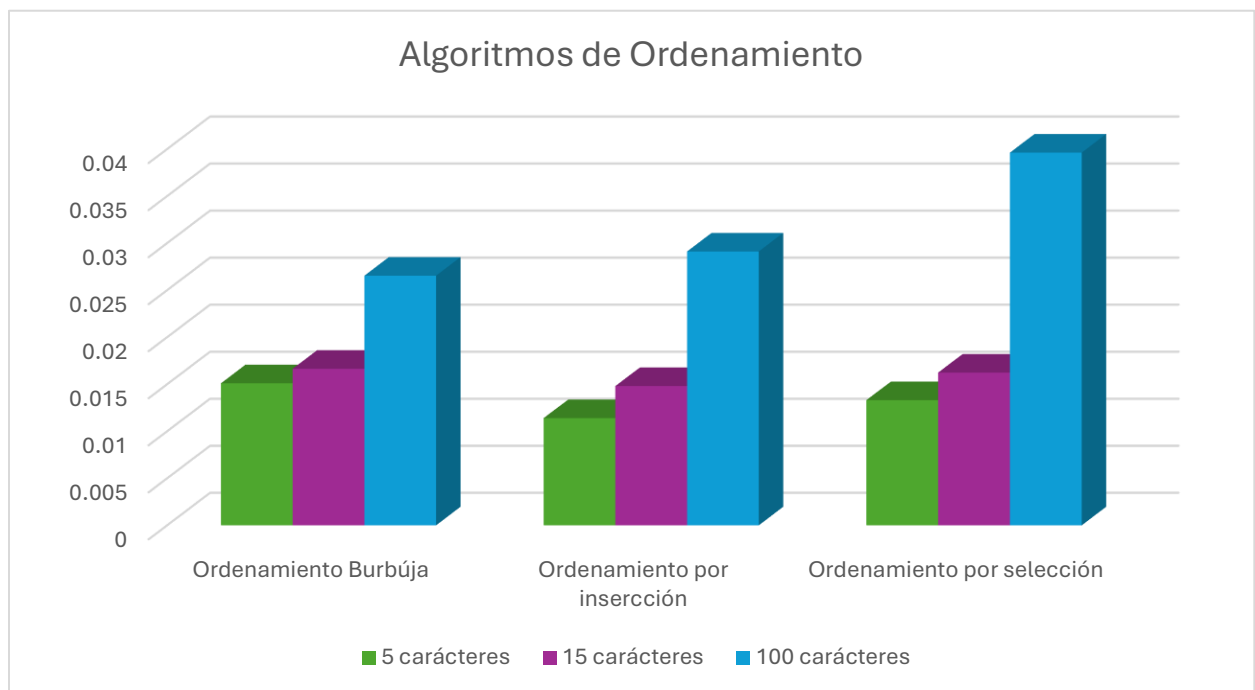
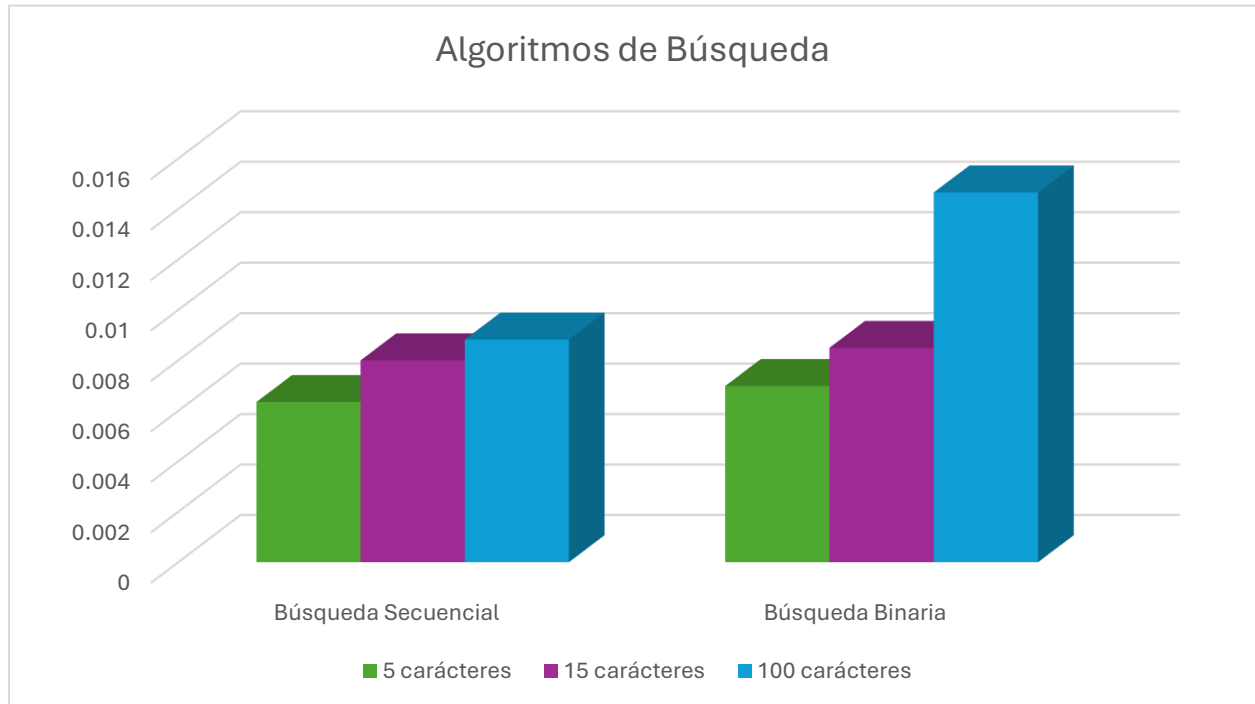
long fin = System.nanoTime();
System.out.println("\nTiempo de ejecución del algoritmo: " + (fin - inicio) + " nanosegundos");
```

Análisis de tiempo de ejecución de cada algoritmo en segundos

Algoritmos de Búsqueda		Conjunto de datos de prueba		
		Pequeños (5 caracteres)	Medianos (15 caracteres)	Grandes (100 caracteres)
	Búsqueda Secuencial	0.0063472 s	0.0079981 s	0.0088221 s
	Búsqueda Binaria	0.006982 s	0.0084929 s	0.0146531 s

Algoritmos de ordenamiento		Conjunto de datos de prueba		
		Pequeños (5 caracteres)	Medianos (15 caracteres)	Grandes (100 caracteres)
	Ordenamiento burbuja	0.0150971 s	0.0166448 s	0.0265351 s
	Ordenamiento por inserción	0.0113939 s	0.0148028 s	0.0291104 s
	Ordenamiento por selección	0.0133069 s	0.0162366 s	0.039618 s

Gráfica para visualizar los resultados



Conclusiones

Cada uno de los diferentes algoritmos tiene sus ventajas y desventajas dependiendo del conjunto de datos que se desean manejar. Gracias a esta práctica y a una investigación puedo concluir lo siguiente para cada algoritmo:

- Algoritmos de búsqueda:
 - Búsqueda secuencial: Este tipo de algoritmo es más recomendado para conjunto de listas con elementos pequeños, cuando las listas están desordenadas. Sin embargo, no es recomendado para listas grandes debido a que no es eficiente porque el tiempo de proceso crece con el tamaño de la lista
 - Búsqueda binaria: Se utiliza con listas que deben de estar ordenadas, lo cual a su vez es una desventaja, porque se necesita implementar otro algoritmo antes de implementar si la lista no está ordenada. Es eficiente con listas grandes y cuando se realizan muchas búsquedas.
- Algoritmos de ordenamiento:
 - Ordenamiento Burbuja: Este algoritmo solamente es eficiente con listas pequeñas, y principalmente es usados para fines didácticos de lógica debido que es muy ineficiente para listas medianas y grandes.
 - Ordenamiento por inserción: Este es un algoritmo útil para listas pequeñas o que casi están ordenadas. Pero no es eficiente con grandes conjuntos de elementos.
 - Ordenamiento por selección: Debido a que este algoritmo siempre tiene el mismo rendimiento, es práctico implementarla cuando los intercambios son costosos pero las comparaciones no lo son.

Recomendaciones

A partir de los datos obtenidos de las pruebas cronometras del tiempo de ejecución de cada tipo de algoritmo, puedo determinar las siguientes recomendaciones para seleccionar algoritmos en situaciones reales:

- Algoritmos de búsqueda:

En escenarios reales, la búsqueda binaria es la mejor herramienta cuando los datos ya están previamente ordenados, por otro lado, la búsqueda secuencial es práctica para estructuras dinámicas o listas cortas por su simple lógica.

Situaciones reales	Algoritmo sugerido	Justificación
Datos pequeños	Búsqueda Secuencial	Su velocidad de ejecución es mejor que la de una búsqueda binaria, y su lógica es mucho más ligera.
Datos medianos sin ordenar	Búsqueda Secuencial	No necesita que los datos estén ordenados previamente como en la búsqueda binaria, por lo que su implementación es más eficaz
Datos medianos ordenados	Búsqueda Binaria	Como los datos ya están ordenados, es un poco más eficiente usar una búsqueda binaria que una secuencial en este caso.
Datos grandes ordenados	Búsqueda Binaria	En grandes ordenados es más rápida que una búsqueda secuencial.
Datos grandes desordenados	Búsqueda Secuencial	No necesita que los datos estén ordenados

- Algoritmos de ordenamiento:

Situaciones reales	Algoritmo sugerido	Justificación
Datos pequeños	Ordenamiento por inserción	Su velocidad es la más alta en las pruebas, y es ideal para listas pequeñas o casi ordenadas
Datos medianos	Ordenamiento por inserción o Burbuja	Ambos tienen similares resultados en las pruebas. El ordenamiento por inserción sigue siendo un poco más eficiente y la burbuja es útil para código que ya está estructurado para comparaciones sencillas
Datos grandes	Ordenamiento por burbuja (en mis pruebas) Ordenamiento por inserción (teórica)	Teóricamente el ordenamiento burbuja es menos eficiente, sin embargo, en las pruebas fue más eficiente que el de selección o inserción, pero esto se pudo haber debido al conjunto de datos probados o al entorno de prueba. Por tanto, siempre se recomienda utilizar algún otro algoritmo de ordenamiento antes que el de burbuja